Articles in this section

Beatsource to Beatport migration

Beatsource to Beatport track mapping API endpoint

Streaming Audio Quality Preference

Authenticating with our Identity Service

How to query search service

Clearing login cache with removeCachedLoginFiles()

Tombstoned Tracks

How to query search service

1 year ago · Updated

This how-to guide is an introduction for those looking to test or implement Beatport's new search service. It will walk you through the process of querying our search service using our swagger docs page which details our search API while conveniently providing a UI to perform such API calls.

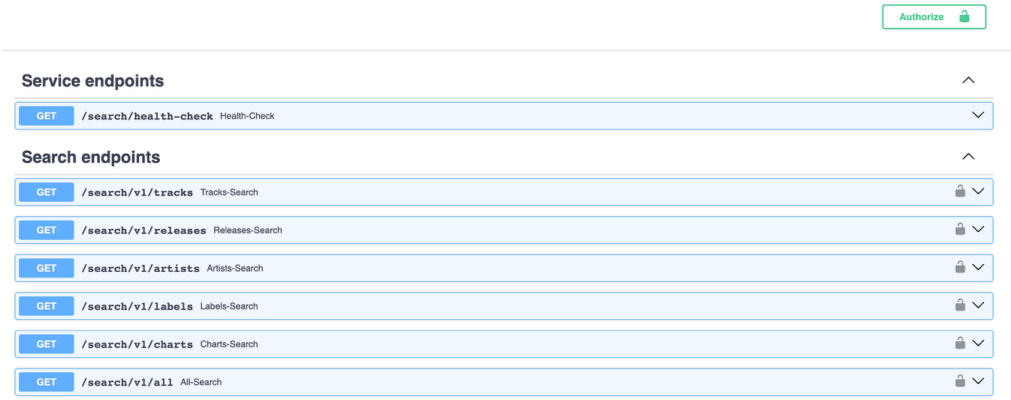
Documentation links and instructions
Query filters
Querying search service via API
Querying search service via the SDK
Resizing image links
Responses

Documentation links and instructions

Beatport - https://api.beatport.com/search/docs

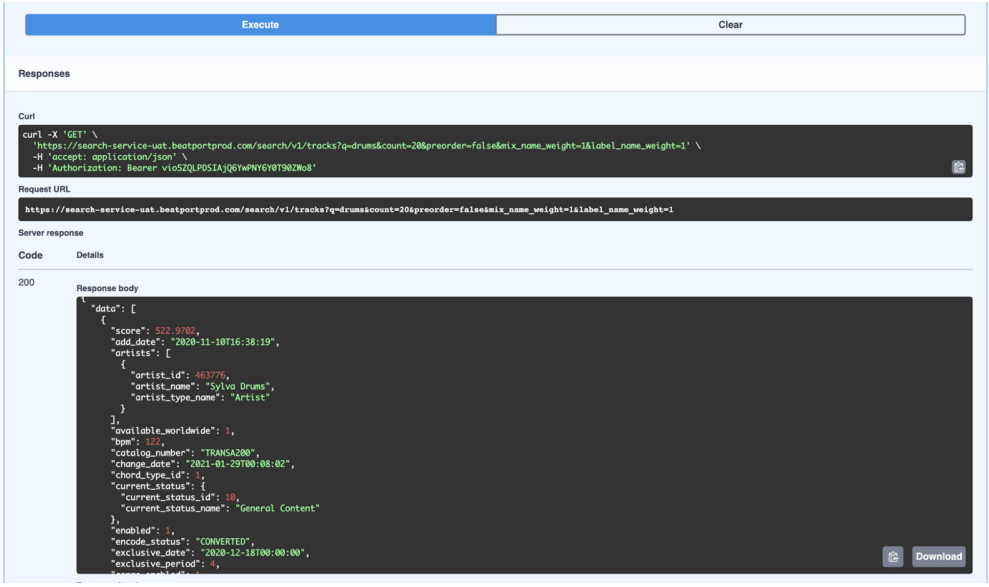
Beatsource - https://api.beatsource.com/search/docs

The search service contains 5 types of indexes: **artists**, **tracks**, **charts**, **labels**, and **releases**. An endpoint is provided for querying each index as well as 1 for querying **all** indexes together. To begin querying these endpoints, head over to the docs page above and you should see an authorize button and all possible routes:



To begin you will a token to authorize, you can find information on our authentication processes in [our API4 documentation](). This is no different than getting a token for your current application, simply call our auth/o/token route to get a token and copy paste it in the authorize box. The scope of your token will have an impact on the results you get, for example, if your scope is app:locker which is strictly for streaming applications, then you will only be returned tracks that are available for streaming.

Once authenticated, to start querying select a route you'd like to test, click try it out, fill out the query param q with a search query, and hit execute to see the results.



Query filters

Each search route offers several filtering options. You can find information on the different filters and their definitions in the search documentation

Beatport - https://api.beatport.com/search/docs
Beatsource - https://api.beatsource.com/search/docs

count integer (query)	The number of results returned in the response 20
preorder boolean (query)	When FALSE, the response will not include tracks in a pre-order status. When TRUE, the response will include tracks that are in a pre-order status false
from_publish_date (query)	The date a track was published on Beatport.com or Beatsource.com. Format: YYYY-MM-DD from_publish_date

Querying search service via API

The root urls for search are

> api.beatport.com/search/v1
>
> api.beatsource.com/search/v1

Each index has its own route followed by the query and filters, for example a call to retrieve 100 Streamable only Beatport tracks with a genre id of 89 and 6 that match the query term Drums would look like

https://api.beatport.com/search/v1/tracks?q=Drums&count=100&genre_id=89,6&is_available_for_streaming=true

You can see examples of the request URL when you execute a call in the documentation

Responses

Curl

```
curl -X 'GET' \
  'https://search-service.beatportstage.com/search/v1/tracks?q=Drums&count=100&genre_id=89%2C6&is_available_for_streaming=true' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer qPAiVT6GNDzJQK7uWaThol3KTB2HKy'
```

Request URL

```
https://search-service.beatportstage.com/search/v1/tracks?q=Drums&count=100&genre_id=89%2C6&is_available_for_streaming=true
```

Querying search service via the SDK

Search service is only available in SDK version 2.7 and above

To query the search service via the Beatport SDK, the search method from beatport::client or beatport::GuestClient must be used.

The signature of the beatport::client::search methods is the following:

```
bool *search(query::callback callback, void *clientdata, searcl
             unsigned int count = 20, const std::map<std::strin
```

The return value is a pointer to a boolean, to cancel the request, the value of a pointer needs to be set to false. This return value is only valid before the callback is called.

Method arguments are:

> callback - results will be received with this callback.

clientdata - custom data passed to the callback.

type - it refers to the type of search (defined in beatport::searchType enumeration).

q - query parameter, this corresponds to 'q' parameter in the search service documentation.

count - a positive integer that represents the number of results to retrieve.

params - Filter parameters, a map of filter parameters will be created, for additional information please refer to the search service docs to see what are possible parameters and their values.

Usage examples with proper callback functions:

```
query_callback = [](beatport::client* client, void* client_data
    beatport::query::status status, const char* response_text,
    const char* next_url, const char*prev_url, int num_results,
    int page_count) {
    // NOTE: in the callback 3rd party applications must handle
};

constexpr int COUNT = 30;

// Search All (with filter parameters)
client->search(query_callback, this, beatport::All, "query stri
        {
            {"enabled", "true"},
            {"available", "true"},
            {"preorder", "false"},
        });

// Search Tracks (with filter parameters)
client->search(query_callback, this, beatport::searchType::Trac

// Search Labels (with filter parameters)
const std::map<std::string, std::string> params{{"enabled", "tr
client->search(query_callback, this, beatport::searchType::Labe

// Search Artists (without filter parameters)
client->search(query_callback, this, beatport::searchType::Arti

// Search Charts (without filter parameters)
client->search(query_callback, this, beatport::searchType::Char
```

The example above shows how to use the beatport::client::search method in various search types, with or without filter parameters, based on your needs.

The response JSON will have a JSON schema that is documented on the search API

docs. To retrieve results from JSON using Beatport SDK JSON parser, the easiest way to do it is to utilize atKey method. Below you can find an example for search type 'ALL':

```
auto tracks   = response_json->atKey("tracks")->arrayAtKey("da
auto releases = response_json->atKey("releases")->arrayAtKey("
auto labels   = response_json->atKey("labels")->arrayAtKey("da
auto artists  = response_json->atKey("artists")->arrayAtKey("d
auto charts   = response_json->atKey("charts")->arrayAtKey("da
```

By looking at the JSON schema for each Search type response, it is easy to transfer that specific data from the Search into the 3rd party application.

Resizing image links

Image links for releases, artists, or labels will be returned with a default size in the URL.
ex: "release_image_uri": "https://geo-media.beatsource.com/image_size/**1400x1400**/c/1/6/c16b2aa8-b952-47ee-a128-a910c183f5fe.jpg"

If the image is too large you can resize it to fit your needs and the image will be altered and delivered with the specified size:

ex: "release_image_uri": "https://geo-media.beatsource.com/image_size/**300x300**/c/1/6/c16b2aa8-b952-47ee-a128-a910c183f5fe.jpg"

Responses

The format of responses from calling the API is detailed in the schemas section found at the bottom of the documentation. This is can be a quick and handy way of checking response formats and their definitions (more to be added).

```
LabelsDefaultModel >

LabelsResponse >

MultisearchResponse v {
  description:        Response model for the all-search endpoint.

  tracks*             TracksResponse > {...}
  artists*            ArtistsResponse > {...}
  charts*             ChartsResponse > {...}
  labels*             LabelsResponse > {...}
  releases*           ReleasesResponse v {
    description:        Response model for the releases endpoint.

    debug               Debug > {...}
    data*               Root list v [
                          title: Root list
                        List of release models.

                        ReleasesDefaultModel v {
                          score*                number
                                                title: Score
                          current_status        Current Status > [...]
                          genre                 Genre > {...}
                          label*                TrackOrReleaseLabelModel > {...}
                          tracks                Tracks > [...]
                          key                   Key > [...]
                          artists               Artists > [...]
                          aggregator*           ReleaseAggregatorModel > {...}
                          available_worldwide*  integer
                                                title: Available Worldwide
                          catalog_number*       string
                                                title: Catalog Number
                          create_date*          string
                                                title: Create Date
                          encoded_date          string
                                                title: Encoded Date
                          exclusive*            integer
                                                title: Exclusive
                          exclusive_date        string
                                                title: Exclusive Date
                          streaming_date        string
                                                title: Streaming Date
                          preorder_date         string
                                                title: Preorder Date
```

Was this article helpful?

Yes No

0 out of 2 found this helpful

Have more questions? Submit a request

Related articles

Authenticating with our Identity Service

Beatport API

How to receive an anon token for Preview Mode and an access token for signed in users

Streaming Audio Quality Preference

Clearing login cache with removeCachedLoginFiles()